ChainLight

Spacecoin Staking Contract Security Audit

: Spacecoin Staking Contract Security Audit

October 22, 2025

Revision 1.0

ChainLight@Theori

Table of Contents

Executive Summary

From September 26, 2025, Theori's ChainLight team conducted a security audit of the Space staking contract over a one-week period. The assessment focused on the staking and reward accounting logic, including staking period updates, interest rate updates, pending stake handling, permit-based staking, and reward settlement flows. During the audit, several issues were identified that could affect reward consistency and system usability, including underflow conditions caused by staking period or interest rate changes, incorrect pending debt accumulation, permit signature front-running, and truncation-related reward calculation failures.

Audit Overview

Scope

Name	Spacecoin Staking Contract Security Audit
Target / Version	• Git Repository (`gluwa/Staking-Contract`): commit `99882c4bce39cc2f1cb7564da44281d828d953f9`
Application Type	Smart contracts
Lang. / Platforms	Smart contracts [Solidity]

Code Revision

N/A

Severity Categories

Severity	Description
Critical	The attack cost is low (not requiring much time or effort to succeed in the actual attack), and the vulnerability causes a high-impact issue. (e.g., Effect on service availability, Attacker taking financial gain)
High	An attacker can succeed in an attack which clearly causes problems in the service's operation. Even when the attack cost is high, the severity of the issue is considered "high" if the impact of the attack is remarkably high.
Medium	An attacker may perform an unintended action in the service, and the action may impact service operation. However, there are some restrictions for the actual attack to succeed.
Low	An attacker can perform an unintended action in the service, but the action does not cause significant impact or the success rate of the attack is remarkably low.
Informational	Any informational findings that do not directly impact the user or the protocol.
Note	Neutral information about the target that is not directly related to the project's safety and security.

Status Categories

Status	Description
Reported	ChainLight reported the issue to the client.
WIP	The client is working on the patch.
Patched	The client fully resolved the issue by patching the root cause.
Mitigated	The client resolved the issue by reducing the risk to an acceptable level by introducing mitigations.
Acknowledged	The client acknowledged the potential risk, but they will resolve it later.
Won't Fix	The client acknowledged the potential risk, but they decided to accept the risk.

Finding Breakdown by Severity

Category	Count	Findings
Critical	**1**	• `SPACESTAKING-003`
High	**3**	• `SPACESTAKING-001` • `SPACESTAKING-002` • `SPACESTAKING-005`
Medium	**0**	• N/A
Low	**1**	• `SPACESTAKING-004`
Informational	**1**	• `SPACESTAKING-006`
Note	**0**	• N/A

Findings

Summary

#	ID	Title	Severity	Status
1	SPACESTAKING-001	Improper staking period update can break pending reward accounting	High	Patched
2	SPACESTAKING-002	Improper interest rate updates can invalidate pending debt accounting	High	Patched
3	SPACESTAKING-003	Incorrect pending debt accumulation in reward calculation inflates rewards	Critical	Patched
4	SPACESTAKING-004	Permit signature can be front-run to disrupt intended staking	Low	Patched
5	SPACESTAKING-005	Integer truncation in reward accounting can cause underflow in reward calculation	High	Patched
6	SPACESTAKING-006	Minor Suggestions	Informational	Patched

#1 SPACESTAKING-001 Improper staking period update can break pending reward accounting

ID	Summary	Severity
SPACESTAKING-001	When additional `pendingStakes` are added through `stake()`, an underflow may occur in `_calculateDebt()` if the reward calculation is performed with an invalid timestamp ordering. This can occur when the `stakingPeriod` is shortened after pending stake entries have already been created, causing the newly derived reward period to become inconsistent with previously recorded stake timing data. As a result, `pendingDebt` may become excessively large, and reward calculation in `claimReward()` may also underflow.	High

Description

The contract allows users to create additional pending stake entries through `stake()`, and the corresponding reward debt is later computed in `_calculateDebt()`. This logic assumes that the timestamps used for reward accrual remain properly ordered across all pending stake records.

However, if the `stakingPeriod` is reduced after pending stakes have already been recorded, the newly derived `startRewardPeriod` may become smaller than the corresponding `firstStakeTimestamp` of existing pending stake data. Under this condition, `_calculateDebt()` may call `_calculateTimePeriod()` with a `timestamp` value smaller than `firstStakeTimestamp`, causing the subtraction to underflow.

```
timePeriod = timestamp - firstStakeTimestamp; // underflow
```

If `timestamp` is smaller than `firstStakeTimestamp`, Solidity arithmetic will underflow, causing the calculated `pendingDebt` to become unexpectedly large. This invalid accounting state may then propagate to `claimReward()`, where reward calculation can also underflow or otherwise fail.

Impact

High

If triggered, this issue can cause `pendingDebt` and reward amounts to become excessively large due to arithmetic underflow. As a result, affected positions may enter an invalid accounting state in which normal staking and reward claiming are no longer possible, effectively causing a denial of service for impacted users.

Recommendation

The reward accounting logic should enforce valid timestamp ordering before performing debt calculations on pending stake entries.

The implementation should be updated to incorporate the following changes:

1. Revert if `startRewardPeriod` is smaller than the relevant `firstStakeTimestamp`.
2. When appending a new pending stake entry, verify that the latest existing `pendingStakes.endOn` is not later than the `endOn` of the newly added pending stake.

Remediation

Patched

The issue was patched by making `updateStakingPeriod()` revert when `latestStartRewardPeriod >= block.timestamp`, thereby preventing a `stakingPeriod` update from causing the new `startRewardPeriod` to become smaller than that of the previous `stakingPeriod`. In addition, if the operator intends to change the `stakingPeriod`, they must first call `updateAllowedStakingStatus(false)` to suspend new staking and then wait until `latestStartRewardPeriod` is earlier than `block.timestamp` before proceeding with the update.

commit: `77fd8c5a68cc04030a476335a64971e32839d7cd`

#2 SPACESTAKING-002 Improper interest rate updates can invalidate pending debt accounting

ID	Summary	Severity
SPACESTAKING-002	The contract calculates `pendingDebt` for each pending stake at staking time using the interest rate schedule that exists at that moment. However, if `updateInterestRate()` is later called and a new interest rate becomes effective before the `startRewardPeriod` of existing pending stakes, the previously stored `pendingDebt` values may no longer match the updated rate schedule. As a result, `stakeInfo[sender].totalDebt` can become greater than the actual accrued rewards, which may cause reward calculation in `_calculateReward()` to underflow and prevent normal reward claiming.	High

Description

When a user stakes while `stakingPeriod > 0`, the contract stores a `PendingStake` entry whose `pendingDebt` is calculated in `_stake()` as follows:

```
pendingDebt: _calculateDebt(amount, senderStakeInfo.firstStakeTimestamp, s
tartRewardPeriod)
```

This means the debt for the pending stake is fixed at staking time based on the interest rate intervals known at that time over the period from `firstStakeTimestamp` to `startRewardPeriod`.

However, `updateInterestRate()` allows the operator to append a new interest rate at any time, and its effective time is always set to the current block timestamp:

```
interestRates.push(_interestRate);
interestChangedTime[index] = uint64(block.timestamp);
```

Because there is no restriction preventing a new interest rate from being introduced before the `startRewardPeriod` of existing pending stakes, a newly added rate may fall within a time range that was already used to calculate stored `pendingDebt`. In that case, the stored debt becomes inconsistent with the updated interest rate schedule.

For example, if a lower interest rate is added after a user has already staked but before that pending stake reaches `startRewardPeriod`, the previously computed `pendingDebt` will remain based on the older, higher rate for the full interval. Later, when `_calculateReward()` computes rewards using the updated rate history, the actual accrued reward may become smaller than the already recorded debt. As a result, `stakeInfo[sender].totalDebt` may exceed the reward amount that should be available for that position.

This inconsistency becomes critical in `_calculateReward()`, where the contract subtracts stored debt and claimed amounts from the calculated reward:

```
reward -= uint128(stakeInfo[sender].totalDebt - totalPendingDebt + stakeIn
fo[sender].totalClaimedReward) - uint128(stakeInfo[sender].totalEarningFor
Unstake);
```

If the stored debt is larger than the actual reward due to a later interest rate insertion, this subtraction may underflow, causing reward claims to revert or otherwise making reward accounting unusable.

Additionally, `_calculateTimePeriod()` was implemented on the assumption that, for the latest rate branch, `timestamp` is not smaller than `interestChangedTime[index]`. If interest rate updates are restricted using a future-effective-time validation mechanism, `_calculateTimePeriod()` should also be updated to safely handle cases where `interestChangedTime[index] > timestamp`.

Impact

High

This issue can cause stored `pendingDebt` values to exceed the rewards actually accrued under the updated interest rate schedule. As a result, reward calculation may underflow in `_calculateReward()`, preventing affected users from claiming rewards normally. In practice, this can place affected positions into an invalid accounting state where normal reward claiming is no longer possible until the inconsistency is resolved.

Recommendation

The implementation should be updated to incorporate the following changes:

1. `_stake()` should store the latest pending stake start time in a global variable such as `latestStartRewardPeriod`.
2. `updateInterestRate()` should be modified to ensure that a newly added interest rate cannot become effective earlier than `latestStartRewardPeriod`.

This ensures that newly added interest rates are applied only after the reward start time of all existing pending stakes, preventing previously calculated `pendingDebt` values from becoming invalid.

In addition, `_calculateTimePeriod()` should be updated to safely handle cases where `interestChangedTime[index]` is greater than `timestamp`, since the current implementation assumes that this ordering does not occur.

Remediation

Patched

commit: `f3fdee7402acb06b75e218a63b9f618a1ae7ff69`

#3 SPACESTAKING-003 Incorrect pending debt accumulation in reward calculation inflates rewards

ID	Summary	Severity
SPACESTAKING-003	In `_calculateReward()`, `totalPendingDebt` is incorrectly increased inside the interest rate loop even though the full pending debt has already been computed by `_getEffectiveStakedAmountAndPendingDebt()`. As a result, `totalPendingDebt` is added repeatedly for each interest rate entry, causing the final reward amount to be overstated by `pendingDebt * (interestRates.length - 1)`.	Critical

Description

The `_calculateReward()` function retrieves both the effective staked amount and the total pending debt by calling `_getEffectiveStakedAmountAndPendingDebt()`:

```
(uint256 effectiveStakedAmount, uint256 pendingDebt) = _getEffectiveStaked
AmountAndPendingDebt(sender, currentTimestamp);
```

The returned `pendingDebt` value already represents the full sum of `pendingDebt` across all currently pending stakes. However, `_calculateReward()` then enters a loop over all interest rate entries and performs the following operation during each iteration: `totalPendingDebt += pendingDebt;`

Because `pendingDebt` has already been fully aggregated before the loop, adding it again in every iteration causes `totalPendingDebt` to be counted multiple times. Since the loop runs once for each element in `interestRates`, the final value of `totalPendingDebt` becomes inflated by an additional `pendingDebt * (interestRates.length - 1)`.

This inflated `totalPendingDebt` is later used in the reward deduction formula:

```
reward -= uint128(stakeInfo[sender].totalDebt - totalPendingDebt + stakeIn
fo[sender].totalClaimedReward) - uint128(stakeInfo[sender].totalEarningFor
Unstake);
```

As a result, the subtraction is reduced more than intended, which causes the final reward amount to be larger than it should be.

Impact

Critical

This issue causes rewards to be overstated for users with pending stakes whenever multiple interest rate entries exist. The amount of inflation increases with the number of elements in `interestRates`, which can lead to systematic overpayment of rewards and incorrect reward accounting.

Recommendation

The implementation should be updated to avoid re-adding already aggregated pending debt inside `_calculateReward()`.

Remediation

Patched

commit: `4e7a72e505c7f44b07e1b082503f6211ff41c94b`

#4 SPACESTAKING-004 Permit signature can be front-run to disrupt intended staking

ID	Summary	Severity
SPACESTAKING-004	The permit signature used in `stakeWithPermit()` can be front-run by an attacker. A malicious actor may submit the same valid signature before the intended user transaction and call `stakeWithPermit()` with `amountToStake` set to `0` or a minimal value. As a result, the signature is consumed while the user's intended staking fails.	Low

Description

`stakeWithPermit()` accepts a permit signature and immediately uses it to call `permit()` before executing `_stake()`. Because the signature is visible in the mempool, a malicious actor can copy it and front-run the original transaction.

Since the permit signature only approves token allowance and is not bound to the intended staking amount, the attacker can call `stakeWithPermit()` with a different `amountToStake`, including `0` or a very small value. Once the signature is consumed, the original user transaction may fail because the permit nonce has already been used.

Impact

Low

An attacker can consume a user's permit signature before the intended transaction is executed, causing the staking transaction to fail or stake an unintended amount. While this does not directly lead to fund theft, it enables a denial of service against permit-based staking.

Recommendation

The implementation should be updated to prevent permit signatures from being used independently of the intended staking action. In particular, the contract should reject zero-value staking and ensure that the permitted amount is properly bound to the staking amount.

Alternatively, a separate signed authorization scheme may be introduced to explicitly bind the user's signature to the staking parameters.

Remediation

Patched

The issue was patched by modifying `stakeWithPermit()` to accept a single `amount` parameter and use that same value for both `permit()` and `stake()`. As a result, an attacker can no longer call `stakeWithPermit()` with a staking amount different from the permitted amount. However, front-running of the signature via a direct call to `token.permit()` remains possible. The team acknowledged this as an acceptable limitation of the underlying permit mechanism.

commit: `5640e94136573e0ee1511d20abb7c47ecd135c88`

#5 SPACESTAKING-005 Integer truncation in reward accounting can cause underflow in reward calculation

ID	Summary	Severity
SPACESTAKING-005	Integer truncation in `_calculateReward()` and `_calculateDebt()` can cause previously stored accounting values to become larger than the newly calculated reward. As a result, the subtraction in `_calculateReward()` may underflow, making reward claims fail.	High

Description

The contract uses integer division in both `_calculateReward()` and `_calculateDebt()`, which causes fractional values to be rounded down. Because previously stored accounting values such as `totalClaimedReward` and `totalEarningForUnstake` are later compared against newly calculated rewards, these rounding effects can become inconsistent across `claimReward()` and `unstake()`.

In `_calculateReward()`, the reward is calculated as follows:

```
reward += uint128(
    effectiveStakedAmount * _calculateTimePeriod(i, currentTimestamp, firs
tStakeTimestamp)
    * interestRates[i] / RATE_DENOMINATOR / 365 days
);
```

Similarly, in `unstake()`, the contract updates `totalEarningForUnstake` using `_calculateDebt()`:

```
stakeInfo[user].totalEarningForUnstake += uint128(
    _calculateDebt(unstakedAmount, userStakeInfo.firstStakeTimestamp, curr
entTimestamp)
);
```

Because both calculations round down, the same stake position may produce different truncated results depending on the amount being used in each step.

For example, assume the interest rate is `0.1 * RATE_DENOMINATOR` and a user stakes `20` tokens for `365 days`. In that case, `claimReward()` computes the reward as follows:

```
20 * 365 days * 10_000 / 100_000 / 365 days = 2
```

As a result, `stakeInfo[sender].totalClaimedReward` becomes `2`.

If the user then unstakes `1` token, `_calculateDebt()` for that unstaked amount may be rounded down to `0`:

```
1 * 365 days * 10_000 / 100_000 / 365 days = 0
```

After that, the user's effective staked amount becomes `19`, and a later call to `claimReward()` may calculate:

```
19 * 365 days * 10_000 / 100_000 / 365 days = 1
```

At this point, `_calculateReward()` performs the following subtraction:

```
reward -= uint128(stakeInfo[sender].totalDebt - totalPendingDebt + stakeIn
fo[sender].totalClaimedReward)
          - uint128(stakeInfo[sender].totalEarningForUnstake);
```

Under the above scenario, this effectively becomes:

```
1 -= (0 - 0 + 2) - 0
```

Since the subtraction is performed inside an `unchecked` block, it underflows instead of reverting safely. As a result, reward claiming may fail because previously stored accounting values become larger than the newly calculated reward due solely to truncation.

Impact

High

This issue can cause `claimReward()` to revert due to arithmetic underflow. As a result, affected users may be unable to claim rewards normally, and the staking position may enter a broken accounting state.

Recommendation

Remove the `unchecked` block in `_calculateReward()` and ensure rounding inconsistencies cannot cause stored accounting values to exceed newly calculated rewards.

Remediation

Patched

commit: `4e7a72e505c7f44b07e1b082503f6211ff41c94b`

#6 SPACESTAKING-006 Minor Suggestions

ID	Summary	Severity
SPACESTAKING-006	The description includes multiple suggestions for preventing incorrect settings caused by operational mistakes, mitigating potential issues, and improving code maturity and readability.	Informational

Description

1. The `stakeWithPermit()` function should also perform the validation `if (stakeInfo[owner].isLocked) revert AccountIsLocked();`.
2. In `_unstakeFromPendingStakeQueue()`, replace `userStakeInfo.totalEarningForUnstake += earningForUnstake;` with `userStakeInfo.totalDebt -= earningForUnstake;` and replace `userStakeInfo.totalEarningForUnstake += uint128(pendingStake.pendingDebt);` with `userStakeInfo.totalDebt -= uint128(pendingStake.pendingDebt);`. The reward calculation remains the same before and after these changes, but it is more appropriate to increase `totalEarningForUnstake` only when it reflects rewards that were actually earned.
3. In `_stake()`, cache the result of `_calculateDebt(amount, senderStakeInfo.firstStakeTimestamp, startRewardPeriod)` and reuse it to avoid duplicate calculations.
4. In the `_claimUnstakedAmount` function, `uint32 next = request.next;` is unnecessary. Remove it and change the `current = next;` to `current = request.next;`.

Impact

Informational

Recommendation

Consider applying the suggestions in the description above.

Remediation

Patched

Minor issues 1 and 2, excluding issues 3 and 4 which are related to gas optimization, have been patched as recommended.

Revision History

Version	Date	Description
1.0	October 22, 2025	Initial version

Theori, Inc. (“We”) is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.